
Tetristributed: Cooperative Multiplayer Tetris

Dries Rooryck
Harvard University
Cambridge, MA

dries_rooryck@college.harvard.edu

Joseph Oronto-Pratt
Harvard University
Cambridge, MA

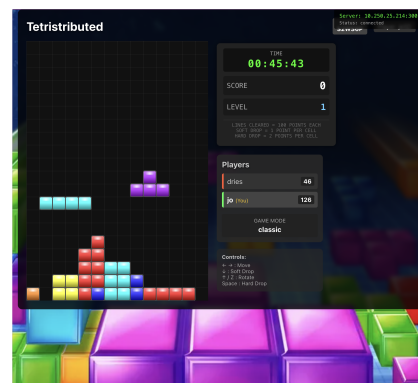
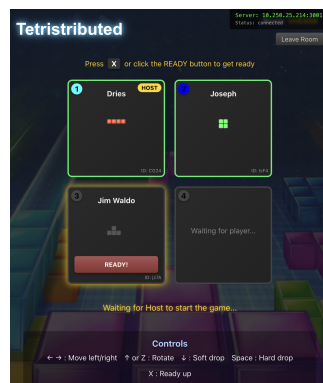
jorontopratt@college.harvard.edu

1 Intro

Tetris is a puzzle video game created in 1985 in which falling tetromino shapes must be neatly sorted into a pile; once a horizontal line of the game board is filled in, it disappears, granting points and preventing the pile from overflowing. Generally, Tetris is a single-player game, where the objective is to keep playing for as long as possible, not allowing your pile to overflow. Tetris was first created by Alexey Pajitnov, a Russian software engineer working at the Dorodnitsyn Computing Centre of the Soviet Academy of Sciences in Moscow. Pajitnov developed the first version of the game on an Electronika 60, a Soviet computer, as a side project while testing the capabilities of the hardware. He wrote Tetris using Pascal for the RT-11 operating system. The game most people know was the version he ported to the IBM PC using Turbo Pascal, and subsequent versions adapted for the Nintendo GameBoy and other gaming consoles. (Source: Wikipedia)

2 What it does (Abstract)

Tetris is a universally recognized and deceptively simple game. We aimed to build a co-op multiplayer version over the network that we could deploy to production. In our version, multiple sets of tetrominoes fall simultaneously, and each player controls one of the sets of blocks on his/her local machine. The project forced us to tackle many distributed systems challenges, from the client synchronization issues of sharing real-time game state under network latency to the reliability issues of 2-fault-tolerance under server failures. We leverage the model of leader-follower server communication in a cluster with a simple heartbeat election protocol for failover, and use a methodical fine grained board locking system to handle concurrency. We built our app on fullstack JavaScript, using React for frontend UI and Node for the webserver, and enhanced Socket.IO for cluster communication. Our app supports up to four players and up to 10 concurrent games.



3 Game Mechanics

The game mechanics of *Tetris* distributed are faithful by design to and preserve many of the original NES design elements, to give the game an authentic feel.

3.1 Gameplay elements

3.1.1 Board size

The game features a standard 20-row board that dynamically expands horizontally based on player count—starting with the traditional 10 columns for solo play and expanding significantly to accommodate additional players (14 columns for two players, 21 for three, and 28 for four). In multiplayer games, the system spawns each player’s pieces in a different starting spot.

3.1.2 Rotation

The rotation system faithfully reproduces the Nintendo Rotation System (NRS) from the original NES Tetris. Each of the seven classic tetrominoes has four distinct rotational states (0°, 90°, 180°, 270°), and the game intentionally omits modern conveniences like wall kicks to maintain authenticity. When players attempt to rotate pieces, the system validates that the rotation won’t cause collisions with the board boundaries, locked blocks, or other players’ pieces—a particularly important check in the multiplayer setting that we expand on later.

3.1.3 Scoring and Progression

Players earn 100 points for each line cleared. The game rewards tactical movement with soft drops (pressing down) awarding 1 point per cell traveled, while hard drops (pressing spacebar) award 2 points per cell. All scores are tracked individually and displayed to all players in real time.

3.1.4 Timing

The game’s timing mechanics add strategic depth through several carefully calibrated delay systems. When a piece touches the ground or other blocks, a lock delay of 30 frames begins, giving players a brief window to slide or rotate the piece before it locks into place. This delay resets when players move or rotate during this window (up to 15 times), enabling advanced techniques like T-spins and last-second adjustments. After a piece locks, an entry delay of 12 frames creates a brief pause before the next piece spawns.

3.1.5 Movement and Control Systems

The implementation of Delayed Auto-Shift (DAS) enhances control responsiveness in a way that honors classic Tetris. When players hold a direction, the system waits 14 frames before initiating piece movement, then continues moving at a rate of one shift every 2 frames.

3.1.6 Piece Falling

Pieces fall at a rate of one cell every 45 frames. During soft-dropping, pieces accelerate dramatically to one cell every 5 frames, giving players significant control over timing. Hard drops, meanwhile, instantly place the piece at its lowest possible position.

When lines are completed, they flash for 30 frames before being removed. The game ends when any player can no longer spawn a new piece, without being outside of the board. Upon the game ending, the player(s) shown the **sum** of their scores as their final team score (if the game was multiplayer) or their ending score (if the game was single-player).

3.1.7 Various Bug Fixes

Throughout the development process, we spotted and fixed some subtle, nontrivial bugs we noted while playtesting, demoing, and asking friends and members of the Harvard SEAS community to play together as well.

- **Hard Drop Collision Fixes.** Collision logic under normal arrow-key movement between different players' pieces was something we figured out fairly quickly after making that design choice, but hard-drop collisions were a lot more subtle given that it is not a common event for a player to hard drop their piece onto another—at one point hard dropping onto another player's piece would cause that piece to lock in mid air and stay frozen. The fix: enforce the same collision rules regardless of the kind of drop and make sure pieces can only lock onto the board floor or onto other locked pieces.
- **Ghost Block Artifacts.** We also fixed a sneaky issue where ghost blocks appeared after line clears by implementing deep board-state copies server-side so that no stale references remained after updates. We applied this logic so that this was true no matter the nature of a line clear, whether by normal gravity, soft dropping, hard dropping, or any of those causing a single, double, or triple line clear.
- **Overhang Relocking Prevention.** We fixed a subtle yet annoying and unintended problem where visual and sound effects got triggered by overhanging segments of already-locked blocks falling down after line clears from below, by introducing a `clearingInProgressRef` flag to stop blocks from incorrectly triggering lock animations after shifting downward post-line clear.
- **Soft Drop Exploit.** We fixed a scoring bug where rapidly tapping the soft-drop key awarded more points compared to holding the key down.
- **Game Start Logic.** We adjusted and refactored player-ready logic on the home screen under join and disconnect instances to eliminate issues with ghost players and incorrect game-over screens appearing for players. As we detail in Section 4 (How it does it: System Architecture), implementing create/join room logic also helped immensely in this regard.

3.1.8 Design

The visual and audio design of *Tetrisdistributed* ultimately unites classic Tetris aesthetics together with modern enhancements. Most notably, we selected hex color codes and piece styling inspired by both classic NES Tetris and modern iterations such as TGM2 in order to create a balance between classic nostalgia and contemporary design. We implemented polished animations for locking, with blocks tinting briefly upon locking and having a sound effect to confirm the lock even more strongly to players. We are especially proud of our design to have lines clear through an aesthetically pleasing particle explosion system, with configurable particle density (set via `aPARTICLES_PER_BLOCK` parameter) to strike a balance between performance and graphical intensity. We also implemented a rotating set of themed background images that cycle every 30 seconds to maintain visual interest just like TGM2 does with its cycling background rotation during games, and we built in a modern looking score panel and timer system with React components to show both total and individual scores live during the game.

4 How it does it (System Architecture)

4.1 Server Design

The *Tetrisdistributed* server architecture employs a leader-follower model where one server (lowest ID) acts as the authoritative source of game state. To understand that, read the cluster section. But when clients connect, an Express middleware redirects them to the leader server. Then, clients can communicate with the server through WebSocket-based real-time communication with automatic fallback to HTTP long-polling when WebSockets aren't supported by the client's environment. This is essentially bidirectional event-based messaging where both client and server can emit named events (like `gameState`, `playerAction`, `createRoom`), listen for specific events, pass structured data as JSON payloads, ... The leader processes all game logic at 60 FPS. The server has a `rooms` object that is a map from room-codes (random 6-letter/digits) to the `gameState` for that room, the host information, and the other player information. The server has one thread per room. The server waits for all players to be in the 'ready' state through UI interaction, then starts a game Loop in its thread for that room. The figure describes roughly what happens in a game loop.

4.1.1 Concurrency

The server uses JS's single-threaded event loop model with the following asynchronous tasks running concurrently. We don't use multiple threads for processes. Every server is its own process.

1. One main HTTP/Express server thread handling web requests on ports 3001-3003
2. One Socket.IO instance handling WebSocket connections.
3. Multiple interval-based game loops (one per active room).
4. Cluster management, like the heartbeat sending interval, heartbeat monitoring interval, and state replication interval (every 3 seconds).

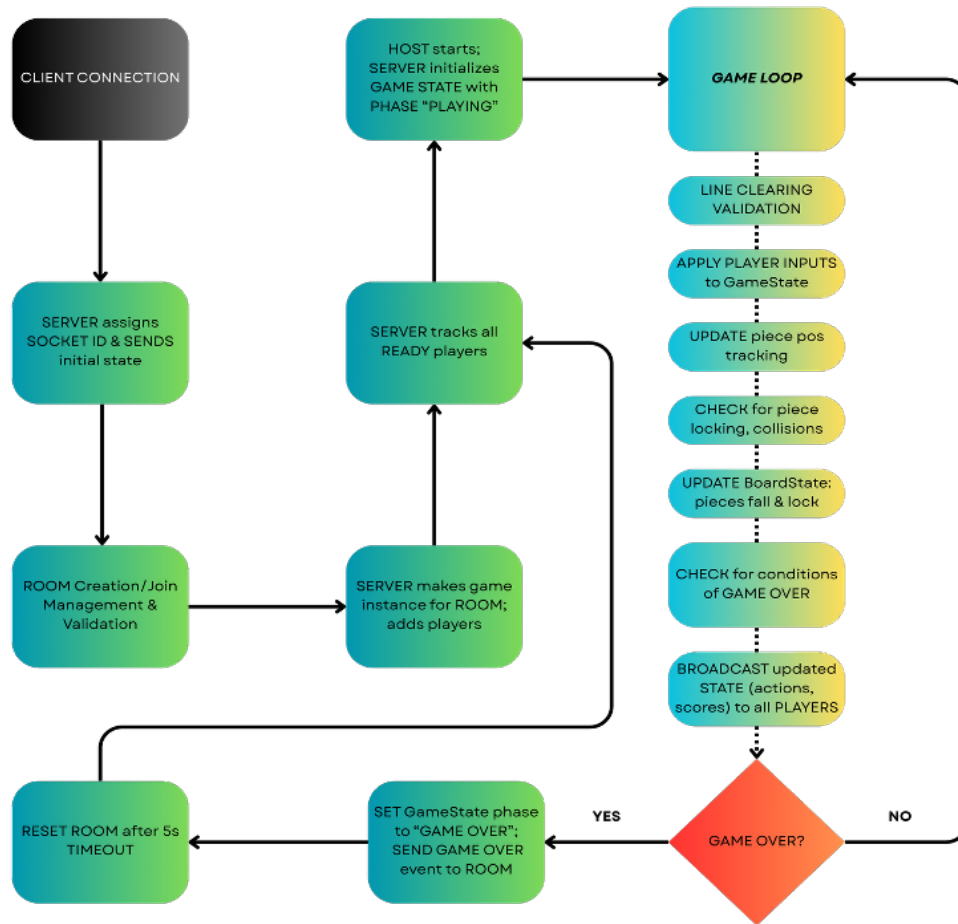


Figure 1: Server flow for handling one game room.

4.2 Cluster Design

4.2.1 How is the leader maintained?

Three server replicas run on unique ports (3001, 3002, 3003) and maintain inter-server communication through dedicated cluster ports (4001, 4002, 4003). All communication is still done with WebSockets. It was an interesting but difficult challenge to make the machinery for cluster management work without using gRPC. The `ClusterManager` class coordinates this distributed system,

implementing heartbeat monitoring, leader election, and state replication on each server across the cluster. When one leader fails, the remaining servers detect this through missed heartbeats for a period of `heartbeatTimeout` and initiate a leader election process (`checkLeadership`) based on server IDs, with the lowest available ID (equivalently, the lowest port) becoming the new leader on each server independently. Only the leader can communicate with the client and only the leader maintains authoritative game state.

4.2.2 How do servers replicate game state?

There are two distinct types of state updates: periodic full state synchronization and real-time incremental updates. Every three seconds, the leader broadcasts a complete snapshot of all active game rooms to all followers through `broadcastFullGameState()`. For real-time gameplay, the leader emits incremental state updates after each frame calculation (approximately 60 times per second, but this is the only way to really have good mid-game failover) via `broadcastState()`, which efficiently transmits only the changed game elements such as tetromino positions, line clears, and score changes.

4.2.3 How does the client find the right server?

Both clients and servers share responsibility. The client is responsible for initial server discovery using its config list, for reconnection by pinging servers when disconnected, for browser redirection to working servers (through its `ServerConnectionManager`, which detects the leader and connects to it), and for session persistence when rejoining games (it maintains a local file with the room number and its own name to 'prove' to the server that it should be reconnected). The server is responsible for HTTP redirects of web clients to the leader, for leader election among server instances, for state replication between leader and followers, and for room recovery when players rejoin on different servers.

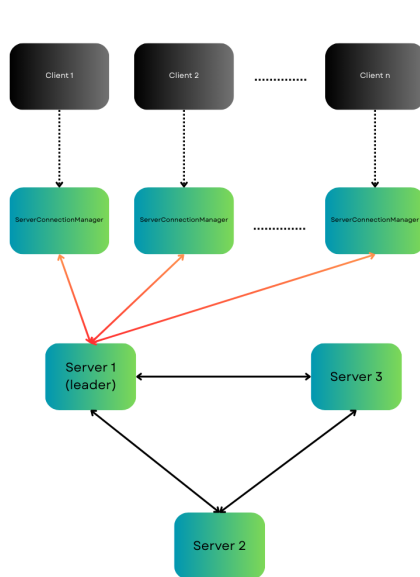


Figure 2: Client-cluster communication.

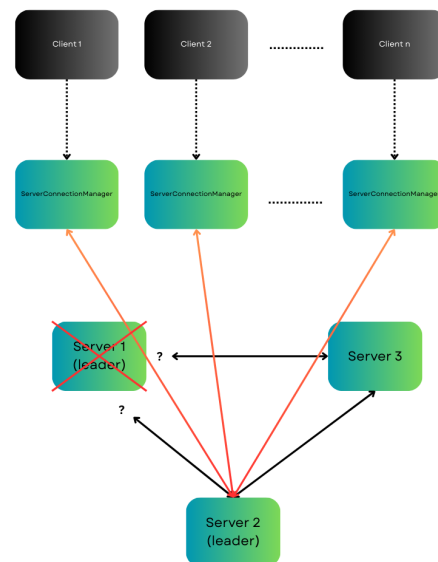


Figure 3: Server 1 dies. Each server independently picks server with lowest port number as successor leader. 2 becomes new leader. ServerConnectionManager picks up on the change and finds new leader to exchange messages with over SocketIO.

4.3 Frontend and UI

The frontend of Tetristributed was built using React, a declarative JavaScript library that allowed us to manage complex state transitions smoothly across the application. For rendering the game board and implementing advanced animations, we used React Konva, a React wrapper for the HTML5 canvas library Konva.js. While new to us, this went a long way in helping us develop a pixel-precise canvas-based rendering pipeline directly within our React components. Konva was essential for implementing visual effects such as block locking tints, line-clearing particle explosions, and real-time rendering of player tetrominoes at 60 FPS. React's component structure also allowed us to modularize the UI into reusable parts like the HomeScreen, ScorePanel, and GameOverScreen, with each part managing its local state while subscribing to global game state updates via WebSocket events.

As for the UI, we deliberately preserved the clean and minimalist aesthetic of classic Tetris, while enhancing it with modern UI responsiveness. The HomeScreen allows players to join and ready up with clear visual feedback. In-game, a rotating set of themed background images adds dynamism, while a prominent score panel and countdown timer keep players informed. The game also differentiates each player with distinct spawn zones and tetromino colors, making it easy to follow each individual's actions during chaotic multiplayer gameplay. Hence, the use of React and Konva provided the flexibility and performance needed to craft an intuitive, visually engaging frontend that supports seamless interaction in real-time distributed gameplay.

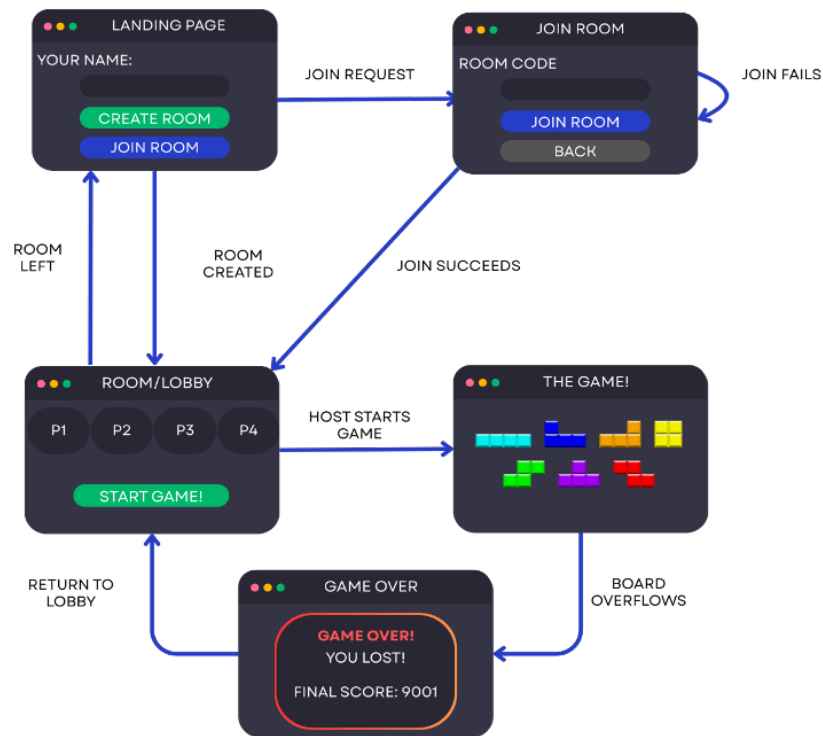


Figure 4: The frontend sequencing that our Tetris game clients get to work with.

5 Testing

Our testing strategy for Tetristributed employed a comprehensive multilayered approach essential for distributed systems. Testing focused not just on functionality (as with unit or integration tests) but also on observing our architecture's reliability under factors like expected network conditions and simulated failures.

5.1 Test Architecture

We organized our tests into a clear hierarchy based on scope and complexity:

1. **Unit tests:** Fast, focused tests for individual components
2. **Integration & end-to-end tests:** Testing communication between components and testing user workflows across the entire system
3. **Fault injection tests:** Deliberate server failures to validate recovery
4. **Load tests:** Testing how much concurrent user pressure is supported before performance takes a hit

We used Jest as our primary test runner with React Testing Library for component tests gave us a consistent testing approach throughout the codebase. We leveraged Socket.IO mocks for client-side tests and dynamic port assignment for server tests to prevent conflicts during parallel test execution.

5.2 Unit Testing

Our unit tests cover core gameplay logic and state management; some critical components we tested include game state logic like independently testing tetromino movement, rotation, collision detection, and line clearing; room management logic like creating rooms and handling player connections and disconnections; leader election, and board state mutations to test and verify that tetromino placements and line clears actually follow game rules.

Additionally, we unit tested React components and custom hooks:

1. **UI components.** Testing rendering logic for game board, tetrominoes, and player UI
2. **Game session hooks** Verifying client-side state management and storage
3. **Server communication.** Mocking Socket.IO to test event handling

5.3 Integration Testing

We also composed integration testing for validating the communication between system components. We focused on:

1. **Socket event handling:** Testing bidirectional Socket.IO communication between client and server
2. **Game flow:** Verifying complete game workflows from room creation through gameplay
3. **Server-to-server communication:** Testing leader-follower heartbeats and state replication

For client-server integration, we created mock servers that implemented the same Socket.IO interface, allowing us to simulate network communications without starting actual servers. This approach enabled us to test the full event cycle wherein the client emits event to server (e.g., `createRoom`), server processes event and updates state, server emits response event to client (e.g., `roomCreated`), and then the client updates its state based on server response. The single most challenging integration tests involved synchronizing multiple "players" (test clients) interacting with the same game room simultaneously (i.e., testing complete multiplayer game flow in its entirety: placing pieces down and having all players' views of the board be the same, concurrent piece movements and collisions, line clearing from one player affecting all players, etc). In a similar yet unique vein, we also wrote end-to-end tests that helped us validate expected game behavior by simulating actual user behavior across the entire system.

To validate our system's 2-fault tolerance, we implemented fault injection tests to automate the action of deliberately crashing servers during active games. These tests

1. Started a 3-server cluster with active game rooms
2. Terminated the leader server using a specialized `/kill-server.sh` script
3. See whether follower servers detected the failure through missed heartbeats

4. Elect a leader and confirm a new leader was properly elected
5. Check that client reconnect to the new leader
6. Check that the game state persisted

Fault injection tests (both automated and manual, i.e. done on our own part) highlighted perhaps one of our most significant challenges: ensuring the new leader server did not misinterpret client reconnections as player disconnections, which would in turn trigger room cleanup. We detail our progress and reflection on this (since we proposed this as a stretch goal) in our Conclusion.

5.4 Load Testing

To evaluate system performance under pressure, we used **Artillery** to simulate high concurrent user loads per the below:

1. Created a phased load test with warm-up, sustained loads, peak loads, and ramp-down phases
2. Attempted to simulate hundreds concurrent client connections
3. Measured connection success rates, response times, and error patterns
4. Monitored server resource utilization during peak loads

Analysis showed that the WebSocket handshake process was a bottleneck, with the server unable to complete connection establishment for super high volumes of incoming requests. We found that our architecture comfortably supported around 10 concurrent games before starting to experience some hiccups in performance.

Testing **Tetristributed** in this comprehensive, multilayered manner had its challenges, but was worth it, as it forced us to learn and problem-solve in addressing careful test synchronization in game loops and network events, avoiding port conflicts with test and production environments, and determining how to simulate realistic (and produce actual) network conditions and server failures.

6 Spinning Up Our Project

6.1 .readme

Note that this information can also be found in our `readme.md` documentation file. In essence, to spin up the server cluster on your local machine that runs **Tetristributed**, simply navigate to the `server` directory and run the shell script `./run-cluster.sh`. You should see a startup process and then some heartbeat communication. Then, in the `client` directory, either run `npm start` to be redirected to a web browser where you can play the game, or navigate to `<servermachineip>:3000`, which is the React development server which will automatically connect the client with the leader server.

6.2 .codebase

7 Future work

1. **Deployment:** Buying a web-domain and hosting our fun game on the internet, with the front-end on Vercel and our server cluster on Render, for example. It is not trivial to host our backend because it uses websockets so we need to think this through.
2. **Scaling:** adding matchmaking systems for a large userbase, distributing handling different instances of the games to different regional servers. This would relieve the load and bottleneck on the number of clients connected.
3. **Move sequencing:** Trying a different method of sequencing moves using a log and Lamport Clocks.
4. **Decentralization:** As an exercise, try our hand at a decentralized version, where clients communicate peer-to-peer. Similar to our classmates who also worked on a tetris game, though it was very different.



Figure 5: Structure of our codebase in the simplest terms.

5. **Security improvements:** Detecting client tampering & cheating attempts using proof-of-action logs. Preventing anybody from posing as another user and ruining their game by just having their metadata.
6. **Game mechanics improvements:** Implementing piece-drop speed-up over the time of the game (via the Tetris leveling system) + maintaining a persistent store of game records in order to motivate players and teams to surpass their high scores.
7. **Failover improvements:** Using nginx for load-balancing and automatic reconnection rather than having to use ctrl+r.

8 Conclusion: What we learned

8.1 Distributed technical challenges

Ultimately, we learned a lot about how core distributed systems concepts become way more difficult when you want to implement them in production. We touch on two. The first two have to do with synchronization, and the third has to do with the nuttiness of 2-fault-tolerance.

1. **Board synchronization & shared state consistency.** All players interact with a single shared board, so even minor desynchronization can break gameplay. We enforced server-authoritative updates on piece locks, line clears, and board state mutations to have all clients remain in sync. Deciding on a model of locking the board took many days. At first, we wanted to allow players to have their pieces fly through each other in mid-air and then to lock on the ground. This presents a ton of challenges where if the pieces fall to the same spot on the ground before locking, there is no good rule what should happen to the overlapping pieces. We experimented with pushing one piece out of the way, predictive locking mechanisms to that there was some collision closer to the ground, but ended up finding that the most intuitive user-experience and easiest to prevent rollbacks of game state were by enforcing collisions in the air.
2. **UI synchronization with server ticks.** Even with smooth input, UI animations (lock flash, explosions) must reflect actual game logic and not client guesswork. We implemented server-controlled animation states broadcast at 60FPS to clients, with precise timers for line clear animations (30 frames), lock delays, and piece entry timing so all players see the same visual effects synchronized with the authoritative game state. Server ticks, setting a global tickrate of 60FPS, was a key design decision to allow for one source of truth with piece positioning. The model of piece movement is that the client requests a piece movement

from the server, but the server decides on what should happen upon that input and sends the state back to the client to display. This way the server can process whatever needs to happen in a frame and then send that frame's update to all clients simultaneously.

3. **Server failover.** We achieved 90% of our stretch goal that gameplay must persist even in the face of server failure and player disconnection, so we implemented leader-follower replication, plus replication of fine-grained game state and player state, with a `rejoinRoom` event allowing players to continue gameplay. This took over 40 hours to get right, and it still is sometimes buggy. We needed to get the sequencing of events exactly right so that the new leader server does not see the client disconnect as a player disconnect event and accidentally cleans up the whole room in the same way it would for a normal player leaving the game. Instead, it should detect a server failure and go into a special mode where it looks for the client's metadata stored client-side, verifies it, then lets them back into the game room. This failover is pretty robust, even in the odd cases when the room isn't preserved, the player can easily start up a new room, so that a server failure doesn't mean an interruption from Tetristributed.

8.2 Other learnings

1. **Line-clear explosion animations:** Rendering dynamic particle explosions as the line clear animation brought on performance challenges, especially when hundreds of particles need to move smoothly or when a player clears several lines at once. We thus optimized animations using React's `useMemo` and `requestAnimationFrame` to keep fluid 60 FPS gameplay without overwhelming the browser rendering pipeline.
2. **Game design and UX iteration.** We gained valuable experience in iterative game design—especially how minor tweaks in animations, scoring feedback, or visual spacing can significantly improve the player experience. Designing multiplayer gameplay with shared goals required extra care in onboarding (home screen logic), exit flow (game over screens), and consistent feedback (sound cues, flashing blocks). We also developed a stronger understanding and appreciation of how user expectations from classic Tetris titles affect the design of modern adaptations like ours that still long to pay homage to Alexey Pajitnov's creation and the many "old-school" adaptations that followed soon after.
3. **Fullstack development using Node, React, and Konva.** We became much more comfortable working across the stack, especially in managing shared game logic between server and client via Node.js. I learned how to effectively use React state, refs, and component lifecycle hooks for smooth gameplay. Additionally, I became proficient with React Konva, a powerful canvas library, for building performant custom animations like lock effects, ghost blocks, and line-clearing explosions. Integrating real-time Socket.IO messages into this rendering loop was a challenge, but ultimately rewarding, as it brought everything together into a visually polished, network-synchronized game that we were proud to have built, and even prouder to have had others play it.
4. **Testing beyond game logic.** We also gained valuable experience in learning various testing paradigms and techniques beyond the experience we acquired in writing up unit and integration tests during the course. We learned how to use Artillery, a gold-standard open-source load testing tool written in Node.js that is perfect for projects like ours that use WebSocket and Socket.IO protocols. More generally, akin to what we remarked earlier about distributed systems concepts sometimes becoming tougher to implement in production, implementing rigorous testing across all the four main axes we outlined (unit, integration/end-to-end, load, and fault injection) helped us understand even more deeply, beyond the course's design exercises, how important testing is.